

UNIVERSIDAD DE LAS FUERZAS ARMADAS



“ESPE”

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

INGENIERÍA EN ANÁLISIS Y DISEÑO DE SOFTWARE

CARRERA DE INGENIERÍA EN SOFTWARE



NOMBRE:

- Blacio Machuca Julio César
- Sigcha Palma Kenned Jhair
- Solano Sanchez Josue Vital
- Toapanta Jama Alexander Jhosue

NRC: 30724

FECHA: 28/05/2026

DOCENTE: Ing. Jenny Ruiz Robalino

Patrón seleccionado

El patrón que vemos más óptimo para emplearlo, es el builder, ya que aporta las siguientes ventajas, teniendo como base el SRS del proyecto.

Justificación

Presencia de objetos complejos con múltiples atributos

El sistema requiere la creación de varias entidades con numerosos atributos, muchos de ellos opcionales o configurables en distintos momentos:

Entidad	Atributos variables	Evidencia en ERS
Clase	día, hora, duración, cupo máximo, entrenador asignado	RF-ADM-01, CUC-ADM-01
Rutina	ejercicios, series, repeticiones, descanso, nivel, notas	RF-ENT-05, CUC-ENT-02
Membresía	tipo (Básica/Premium), precio, características, vencimiento	RF-ADM-02, CUC-ADM-02
Reporte	tipo (asistencia/finanzas/uso), período, formato (PDF/correo)	RF-ADM-04

Necesidad de construir objetos paso a paso

El SRS describe flujos donde los objetos se construyen de manera progresiva, no todos de una sola vez:

- Un entrenador crea una rutina agregando ejercicios uno por uno (CUC-ENT-02).
- Un administrador configura una clase definiendo primero horario, luego cupo, luego entrenador (CUC-ADM-01).
- Un atleta registra su progreso pudiendo ingresar datos manuales o mediante sincronización automática (CUC-ATL-02).

Builder permite construcción incremental con validaciones en cada paso, algo que un constructor tradicional no puede ofrecer.

Múltiples variantes de un mismo tipo de objeto

El sistema debe manejar diferentes variantes de objetos:

- Membresías: Básica, Premium, posibles planes futuros.
- Rutinas: para distintos niveles (principiante, intermedio, avanzado).
- Reportes: con diferentes filtros y formatos de salida.

Builder permite crear estas variantes reutilizando el mismo proceso de construcción sin duplicar lógica, cumpliendo con el principio abierto/cerrado (abierto para extensiones, cerrado para modificaciones).

Alineación con requisitos no funcionales

El uso de Builder contribuye directamente a cumplir con requisitos no funcionales especificados:

RNF	Descripción	Aporte de Builder
RNF-12	Documentación de requisitos	El código refleja claramente los pasos de construcción
RNF-13	Modularidad	Builder separa la construcción de la representación del objeto
RNF-06	Control de accesos	Permite validar permisos en cada paso de construcción

Conclusiones

El patrón Builder es el más adecuado para IronClad Box porque el sistema requiere construir objetos complejos con múltiples atributos opcionales (clases, rutinas, membresías, reportes). Usar constructores tradicionales generaría código ilegible y propenso a errores. Builder permite la construcción paso a paso con validaciones intermedias, evitando objetos inconsistentes y facilitando la creación de distintas variantes sin duplicar lógica.

Además, Builder mejora la mantenibilidad del código al separar la construcción de la representación final, lo que permite incorporar nuevos requisitos del cliente sin romper lo existente. El patrón Prototype queda descartado porque no hay necesidades de clonación de objetos en el alcance del proyecto.